

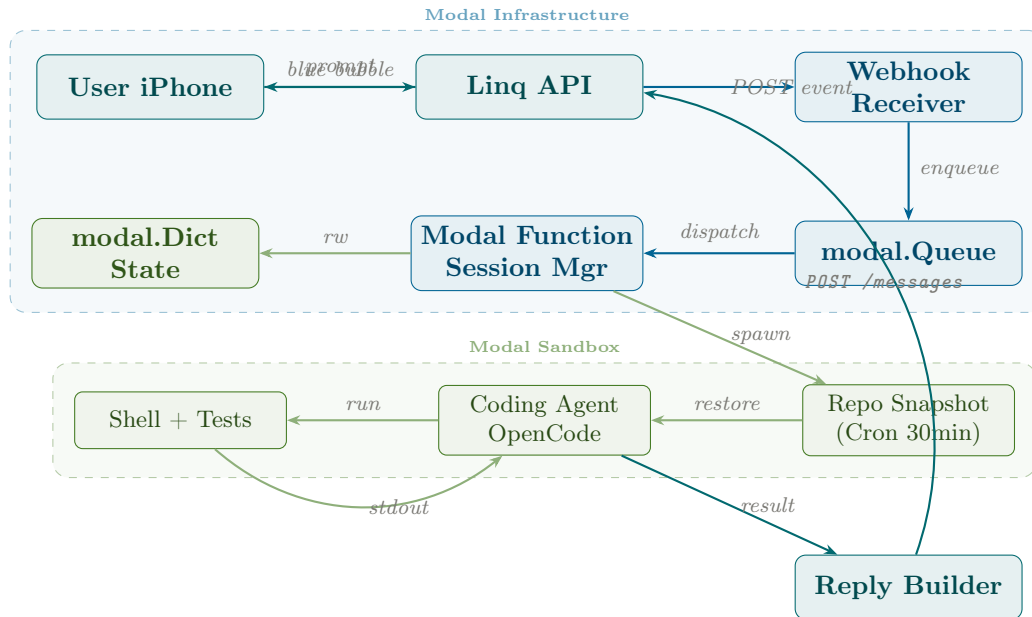
iMessage Coding Agent

Ramp-style Background Agent via Linq API + Modal Sandboxes

Stack: Linq Partner API (iMessage/RCS/SMS) **Compute:** Modal Functions + Sandboxes

Agent: OpenCode / LangGraph

Architecture Overview



Layer 1 — Linq API (iMessage Gateway)

Linq is a SOC 2 Type II certified RESTful API at <https://api.linqapp.com/api/partner/v3> providing programmatic iMessage, RCS, and SMS access with no app install required on the user device.

Authentication

Listing 1: Sending a message via Linq

```
1 curl -X POST https://api.linqapp.com/api/partner/v3/messages \
2   -H "Authorization: Bearer $LINQ_TOKEN" \
3   -H "Content-Type: application/json" \
4   -d '{"to": "+16505550100", "text": "Agent ready.", "protocol": "imessage"}'
```

Inbound Webhook Payload

When a user texts the Linq number, Linq fires a signed POST to your endpoint:

Listing 2: message.received event

```
1 {
2   "event": "message.received",
3   "data": {
4     "id": "msg_01abc...",
5     "from": "+16505550100",
6     "to": "+18005551234",
7     "protocol": "imessage",
8     "text": "Fix the off-by-one in utils.py",
```

```

9     "timestamp": "2026-05-01T15:04:00Z"
10 }
11 }

```

Security: Always verify X-Linq-Signature (HMAC-SHA256) before processing any webhook payload.

Outbound Reply with Threading

Listing 3: Threaded iMessage reply

```

1 import httpx, os
2
3 LINQ_TOKEN = os.environ["LINQ_TOKEN"]
4 LINQ_FROM = os.environ["LINQ_FROM_NUMBER"]
5
6 def send_imessage(to: str, text: str, reply_to_id: str | None = None) -> None:
7     payload = {"from": LINQ_FROM, "to": to,
8               "text": text, "protocol": "imessage"}
9     if reply_to_id:
10         payload["replyToMessageId"] = reply_to_id
11     httpx.post(
12         "https://api.linqapp.com/api/partner/v3/messages",
13         headers={"Authorization": f"Bearer {LINQ_TOKEN}"},
14         json=payload,
15         timeout=10,
16     ).raise_for_status()
17
18 def send_typing(to: str, active: bool) -> None:
19     httpx.post(
20         "https://api.linqapp.com/api/partner/v3/typing",
21         headers={"Authorization": f"Bearer {LINQ_TOKEN}"},
22         json={"to": to, "active": active},
23     )

```

Linq Feature Matrix

Feature	Details
Unified messaging	iMessage, RCS, SMS — auto or explicit <code>protocol</code> field
Rich media	Images, video, voice memos, docs up to 100 MB
Group chats	Create and manage group conversations
Message threading	<code>replyToMessageId</code> field for inline replies
Typing indicators	Show real-time “...” bubble via <code>/typing</code> endpoint
Reactions	Tapbacks + custom Unicode emoji
Message effects	Confetti, fireworks, slam (iMessage only)
Webhooks	Delivery, read receipts, reactions, inbound events
Observability	W3C trace IDs on every request

Layer 2 — Webhook Receiver + Modal Infrastructure

Webhook Receiver (FastAPI)

Listing 4: Verified webhook handler

```

1 import hmac, hashlib, os
2 from fastapi import FastAPI, Request, HTTPException
3 import modal
4
5 app_http = FastAPI()
6 queue = modal.Queue.from_name("linq-prompts", create_if_missing=True)

```

```

7  LINQ_SECRET = os.environ["LINQ_SECRET"].encode()
8
9  @app_http.post("/linq/webhook")
10 async def handle_webhook(request: Request):
11     body = await request.body()
12     sig = request.headers.get("X-Linq-Signature", "")
13     expected = hmac.new(LINQ_SECRET, body, hashlib.sha256).hexdigest()
14     if not hmac.compare_digest(sig, expected):
15         raise HTTPException(status_code=401, detail="Bad signature")
16
17     data = await request.json()
18     if data["event"] == "message.received":
19         queue.put({
20             "sender": data["data"]["from"],
21             "text": data["data"]["text"],
22             "msg_id": data["data"]["id"],
23         })
24     return {"ok": True}

```

Modal Primitives

Primitive	Role
modal.Queue	Decouples Linq webhooks from sandbox execution
modal.Dict	Maps sender phone → sandbox ID + history
modal.Cron	Rebuilds repo snapshot every 30 minutes
modal.Function	Session manager: reads queue, resolves sandbox
modal.Sandbox	Isolated Linux VM with full dev environment
modal.Secret	Secure credential injection into sandboxes

Session Manager Function

Listing 5: Modal session dispatcher

```

1  import modal
2
3  modal_app = modal.App("imessage-coding-agent")
4  queue = modal.Queue.from_name("linq-prompts")
5  sessions = modal.Dict.from_name("agent-sessions", create_if_missing=True)
6
7  @modal_app.function(timeout=86400)
8  def session_manager():
9      while True:
10         job = queue.get(block=True)
11         sender = job["sender"]
12
13         if sender in sessions:
14             sb = modal.Sandbox.from_id(sessions[sender]["sandbox_id"])
15         else:
16             sb = spawn_sandbox()
17             sessions[sender] = {"sandbox_id": sb.object_id, "history": []}
18
19         run_agent.spawn(sb, job["text"], job["msg_id"], sender)

```

Layer 3 — Modal Sandbox Execution

Image + 30-Minute Snapshot Cron

Listing 6: Repo image + cron refresh

```

1  repo_image = (
2      modal.Image.debian_slim()
3      .apt_install("git")

```

```

4     .pip_install("opencode-ai", "pytest", "httpx", "ruff")
5     .run_commands(
6         "git clone https://github.com/your-org/repo /app",
7         "cd /app && pip install -e .",
8     )
9 )
10
11 @modal_app.function(schedule=modal.Cron("*/30 * * * *"))
12 def refresh_snapshot():
13     repo_image.build()    # keeps snapshot under 30 min stale

```

Sandbox Spawn Configuration

Listing 7: Sandbox factory

```

1 def spawn_sandbox() -> modal.Sandbox:
2     return modal.Sandbox.create(
3         image=repo_image,
4         cpu=2.0,
5         memory=4096,          # 4 GB RAM
6         timeout=3600,         # 1-hour session max
7         network_access=True,
8         secrets=[
9             modal.Secret.from_name("openai-key"),
10            modal.Secret.from_name("github-token"),
11        ],
12    )

```

Agent Execution + Streaming Reply

Listing 8: OpenCode execution with streaming iMessage updates

```

1 @modal_app.function()
2 def run_agent(sandbox: modal.Sandbox, prompt: str,
3             msg_id: str, sender: str) -> None:
4     send_typing(sender, active=True)
5
6     proc = sandbox.exec(
7         "opencode", "run",
8         "--prompt", prompt,
9         "--output-format", "stream-json",
10    )
11
12    chunks = []
13    for line in proc.stdout:          # line-by-line stdout stream
14        chunks.append(line.strip())
15
16    proc.wait()
17    result = parse_agent_output(chunks)
18
19    send_typing(sender, active=False)
20    send_imessage(sender, result, reply_to_id=msg_id)

```

End-to-End Data Flow

1. User sends iMessage to the Linq-provisioned phone number.
2. Linq fires POST /linq/webhook with a signed `message_received` event.
3. Webhook receiver verifies HMAC-SHA256 signature, extracts sender + text, and calls `modal.Queue.put(...)`.
4. `session_manager` dequeues the job; looks up or creates a sandbox entry in `modal.Dict`.

5. A Modal Sandbox is spawned (or resumed) from the latest 30-minute snapshot; OpenCode is invoked with the prompt.
6. OpenCode runs shell commands, edits files, and executes tests in a stdout feedback loop until the task is complete.
7. `run_agent` fires a typing indicator via Linq on receipt, then POSTs the final result as a threaded iMessage reply referencing the original `msg_id`.

Configuration Reference

Variable	Set in	Purpose
LINQ_TOKEN	Modal Secret	Bearer token for Linq Partner API
LINQ_SECRET	Modal Secret	HMAC webhook signature key
LINQ_FROM_NUMBER	Env var	E.164 number provisioned by Linq
OPENAI_API_KEY	Modal Secret	LLM calls inside the sandbox
GITHUB_TOKEN	Modal Secret	Repo access for snapshot builds
MODAL_TOKEN_ID	CI/CD env	Modal platform authentication

Key Design Decisions

title= Why Linq over a Mac Mini + AppleScript?

Linq is SOC 2 Type II certified — no persistent Mac hardware, no `chat.db` file hacks, and blue-bubble iMessage delivery at scale. Typing indicators and read receipts are first-class API features.

30-minute snapshot cron Pre-installs all dependencies and pre-clones the repo so sandbox startup stays under 5 seconds (versus minutes from scratch).

Sender → sandbox mapping in `modal.Dict` Each iMessage thread maps 1:1 to a persistent sandbox, giving the agent per-conversation memory with no external database.

Typing indicator on receipt `send_typing(sender, active=True)` is called immediately upon job receipt so the user sees “...” while OpenCode runs.

Threaded replies via `replyToMessageId` All replies reference the original inbound message ID so iMessage groups the entire session into a single thread.

Streaming stdout `sandbox.exec()` stdout is consumed line-by-line, enabling partial progress updates for long-running tasks.